

PERSONAL EXPENSE & BUDGET TRACKER



VITYARTHI Project

Course: Programming in Java

Course Code: CSE2006

Submitted by: Yash Modi

Registration No.: 25BAI11144

Faculty Name: Prof. Dheresh Soni

A) Introduction

Financial literacy and systematic tracking of day-to-day expenditures represent fundamental life skills, particularly for university students and young professionals managing monthly stipends or allowances. The lack of active budgetary tracking frequently precipitates impulsive discretionary purchases, early-month deficit conditions, and financial stress.

The **Personal Expense & Budget Tracker** is a command-line interface (CLI) software solution engineered using Core Java. Designed to run completely offline without external database servers or cloud accounts, the application emphasizes strict data encapsulation, domain-driven package organization, and robust input validation. By integrating real-time budget threshold warnings alongside automated flat-file persistence, the system provides users with immediate spending visibility and discipline.

B) Problem Statement

Modern personal accounting applications often introduce unnecessary barriers to adoption, including mandatory registration, graphical user interface bloat, persistent network dependencies, and third-party data tracking. Conversely, manual tracking via physical notebooks or static spreadsheets suffers from human transcription error and lacks automated budget enforcement.

The primary objective of this project is to implement an offline, resilient, terminal-executable Core Java application that provides:

1. Reliable logging of transactional inflows and outflows with category tagging.
2. Configurable category-level spending limits.
3. Proactive, multi-tier alerting mechanisms prior to budget breaches.
4. Transparent flat-file persistence and exportable statement generation.

C) Functional Requirements

4.1 Module 1: Transaction Engine (CRUD Operations)

- **FR 1.1:** The system shall allow users to log financial transactions categorized as either INCOME or EXPENSE.
- **FR 1.2:** Each transaction must record an automatically generated unique identifier (TXID), timestamp (LocalDate), monetary amount, predefined Category, and descriptive memo.
- **FR 1.3:** The system shall present all recorded transactions in an aligned, readable tabular format via the console interface.

4.2 Module 2: Budget Threshold & Alert Engine

- **FR 2.1:** The system shall permit users to configure monthly budget spending caps for individual

categories.

- **FR 2.2 (Soft Warning):** When an expense causes cumulative spending in a category to reach or exceed 80% of its budget limit, the system must trigger an immediate console warning displaying current utilization percentage.
- **FR 2.3 (Hard Breach Alert):** When an expense causes cumulative spending to exceed 100% of the allocated cap, the system must raise and handle a `BudgetExceededException`, alerting the user of an overdraft.

4.3 Module 3: Analytics & Export Engine

- **FR 3.1:** The system shall calculate Total Inflow, Total Outflow, and Net Balance (*Savings = Inflow – Outflow*) dynamically using Java Stream pipelines.
- **FR 3.2:** The system shall support exporting a timestamped financial balance sheet to `data/summary_report.txt`.
- **FR 3.3:** The repository layer must persist all in-memory records into `data/transactions.csv` dynamically to guarantee survivability across sessions.

D) Non-Functional Requirements

- **NFR 1: Reliability & Fault Tolerance:** File write pipelines utilize atomic stream flushing (`BufferedWriter.flush()`). Data integrity is protected against unexpected crashes, blank strings, and malformed CSV rows through defensive split boundaries (`split(",", -1)`).
- **NFR 2: Usability & User Experience:** The console interface utilizes a structured numerical menu (options 1–6) with clean input prompting, meaningful validation messages, and immediate visual feedback.
- **NFR 3: Maintainability & Modularity:** The codebase adheres to the Single Responsibility Principle (SRP) by separating responsibilities into 5 distinct packages (`exception`, `model`, `repository`, `service`, `ui`), keeping components decoupled and easy to refactor.
- **NFR 4: Performance & Resource Efficiency:** In-memory caching structures (`ArrayList`, `HashMap`) allow transaction insertions, updates, and threshold evaluations to execute in $\mathcal{O}(1)$ to $\mathcal{O}(N)$ computational time without repetitive disk read cycles.

E) System Architecture

The project follows a **Modular Design** pattern, separating concerns into four distinct layers:

1. Presentation Layer (Main.java & MenuUI.java):

- Handles all User Interaction through an interactive CLI Menu.
- Prompts for and sanitizes console inputs (e.g., choice selections, transaction details).
- Formats and displays outputs, tabular statements, alerts, and financial analytics.
- Routes user commands to the appropriate service modules.

2. Business Logic Layer (ExpenseService.java & BudgetService.java):

- Contains core financial computation rules (Total Inflow, Outflow, Net Balance).
- Aggregates spending metrics and monitors category limits against predefined caps.
- Evaluates spending thresholds to issue proactive 80% warnings and 100% breach notifications.
- Enforces business constraints and throws domain-specific checked exceptions (InvalidTransactionException, BudgetExceededException).

3. Data Model Layer (Transaction.java, Budget.java & Category.java):

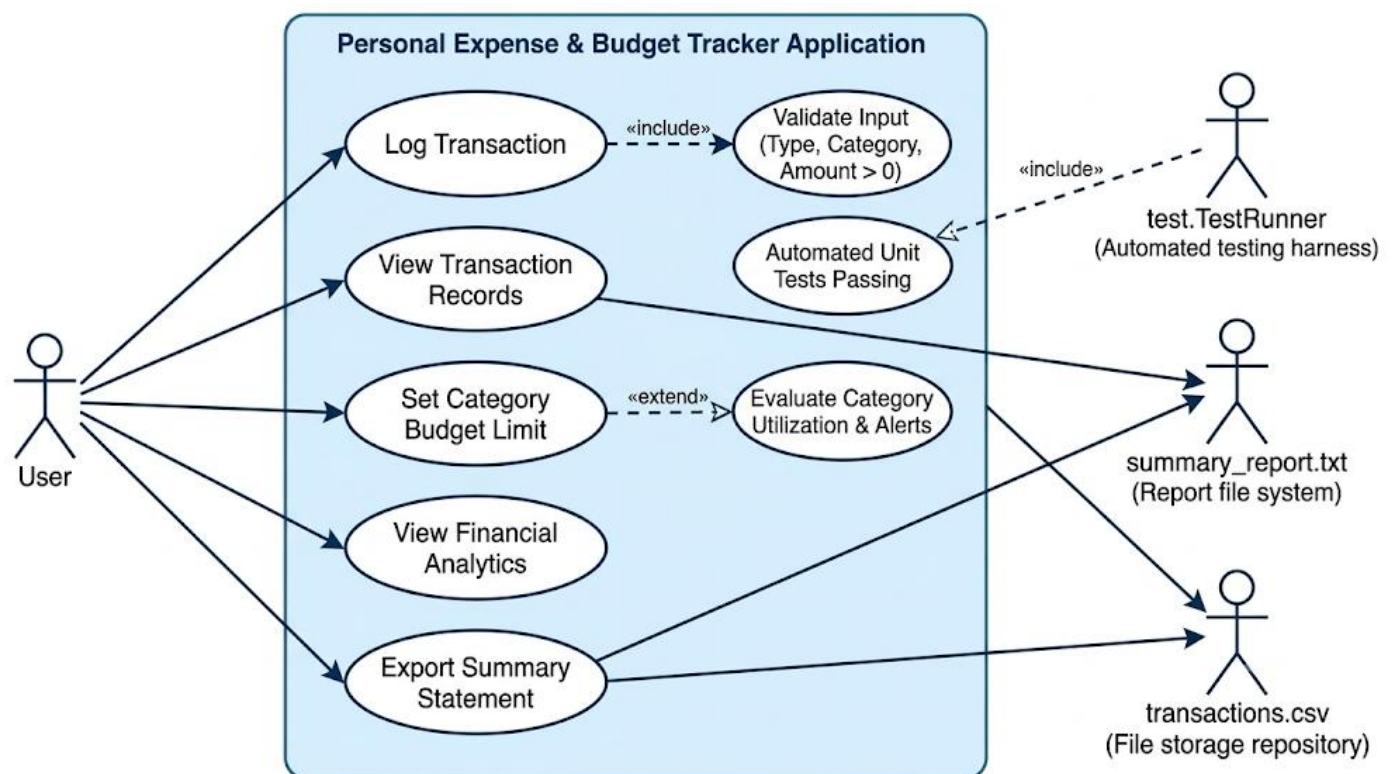
- Defines the core object-oriented structures and domain entities.
- Encapsulates transaction attributes (ID, timestamp, amount, type, memo) and budget caps.
- Provides type safety across transaction categories using an enumeration (Category).
- Handles individual record serialization rules via toCsv() string formatting.

4. Data Persistence Layer (FileStorageRepository.java & transactions.csv):

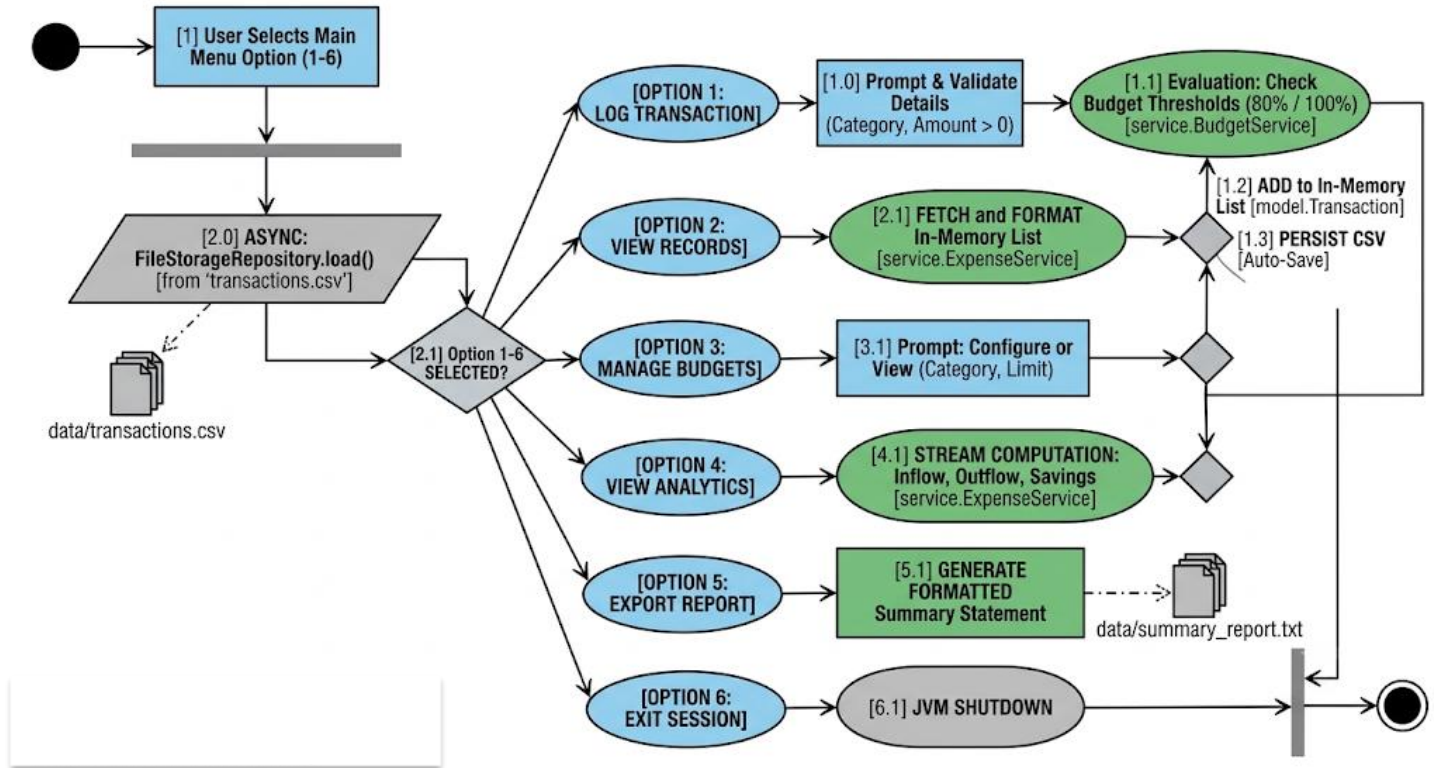
- Manages flat-file I/O (Input/Output) operations using BufferedReader and BufferedWriter.
- Serializes in-memory Transaction object lists into CSV format for persistent storage.
- Deserializes CSV records safely back into Java objects upon application launch.
- Generates and writes formatted text statements to summary_report.txt.

F) Design Diagrams

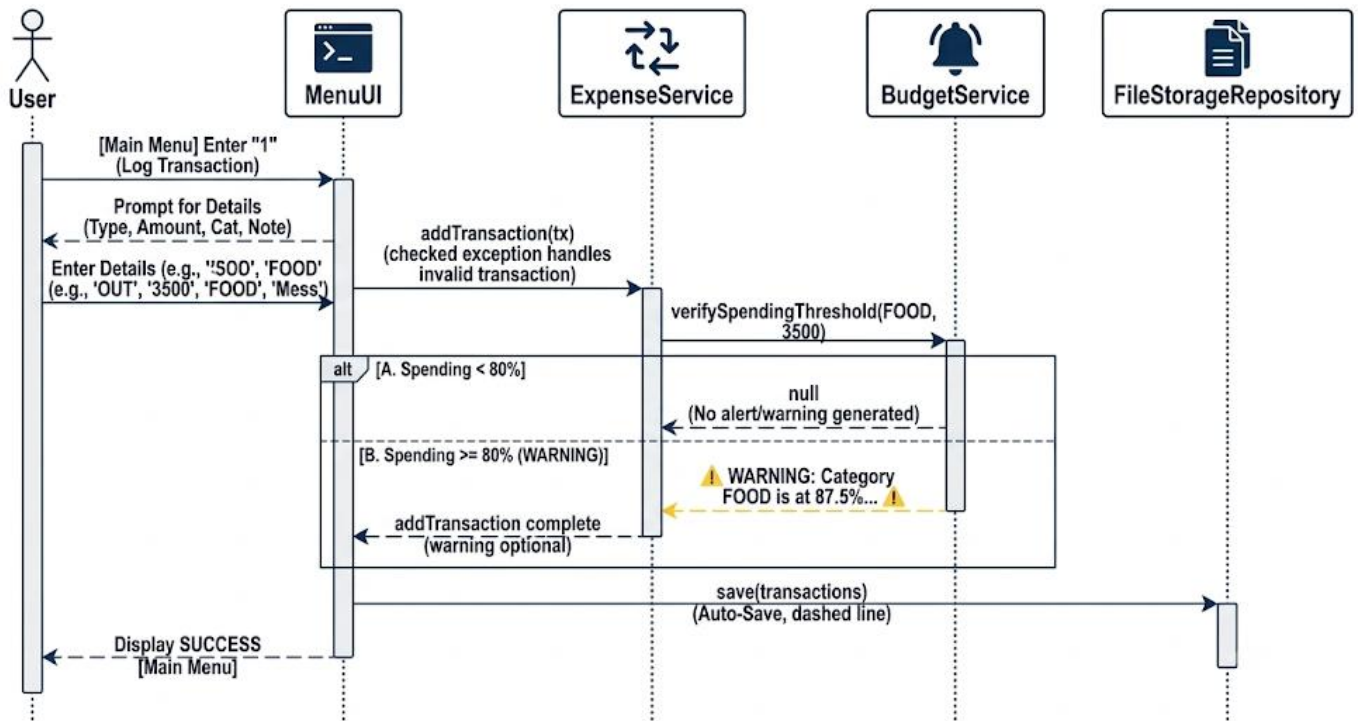
1. Use Case Diagram



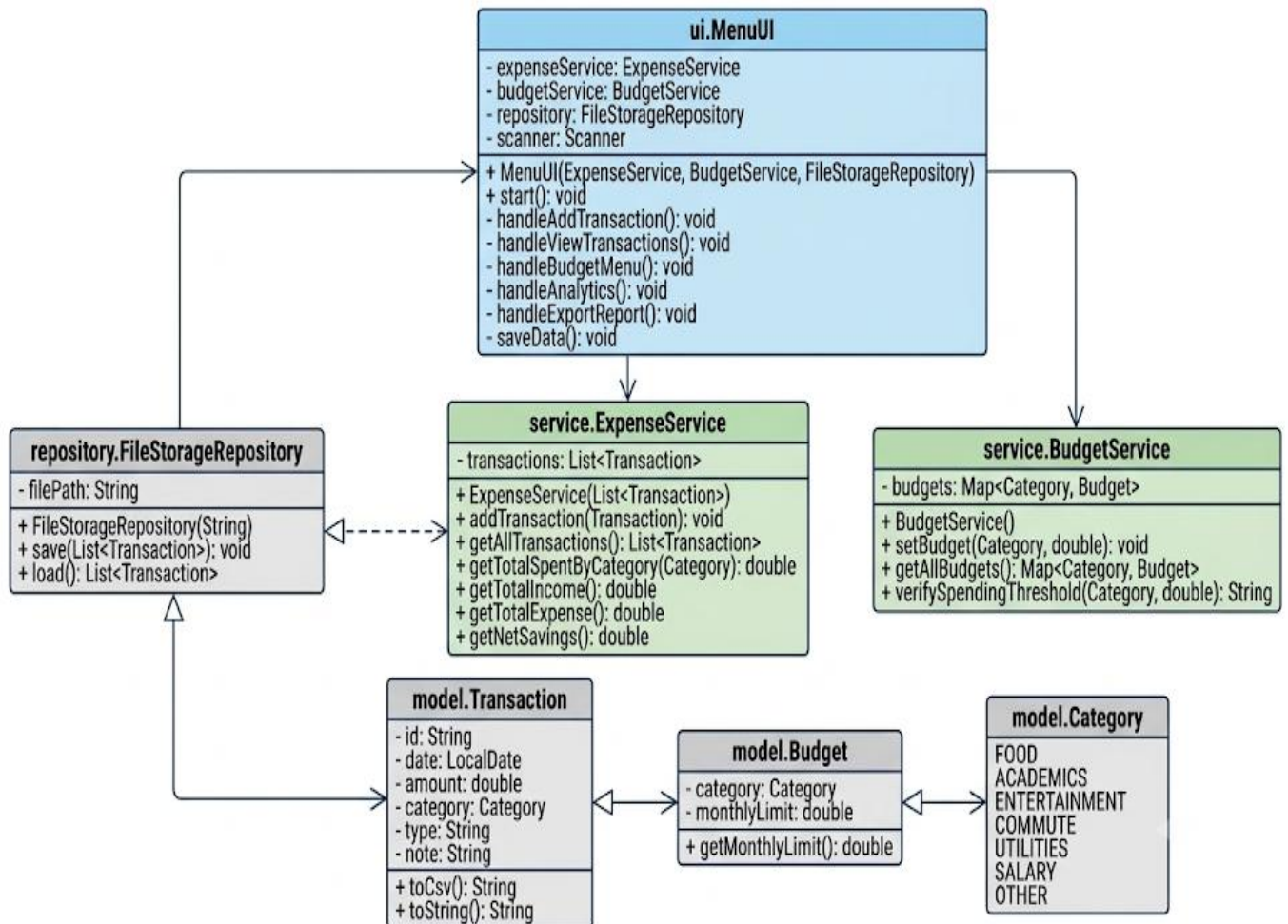
2. Workflow Diagram



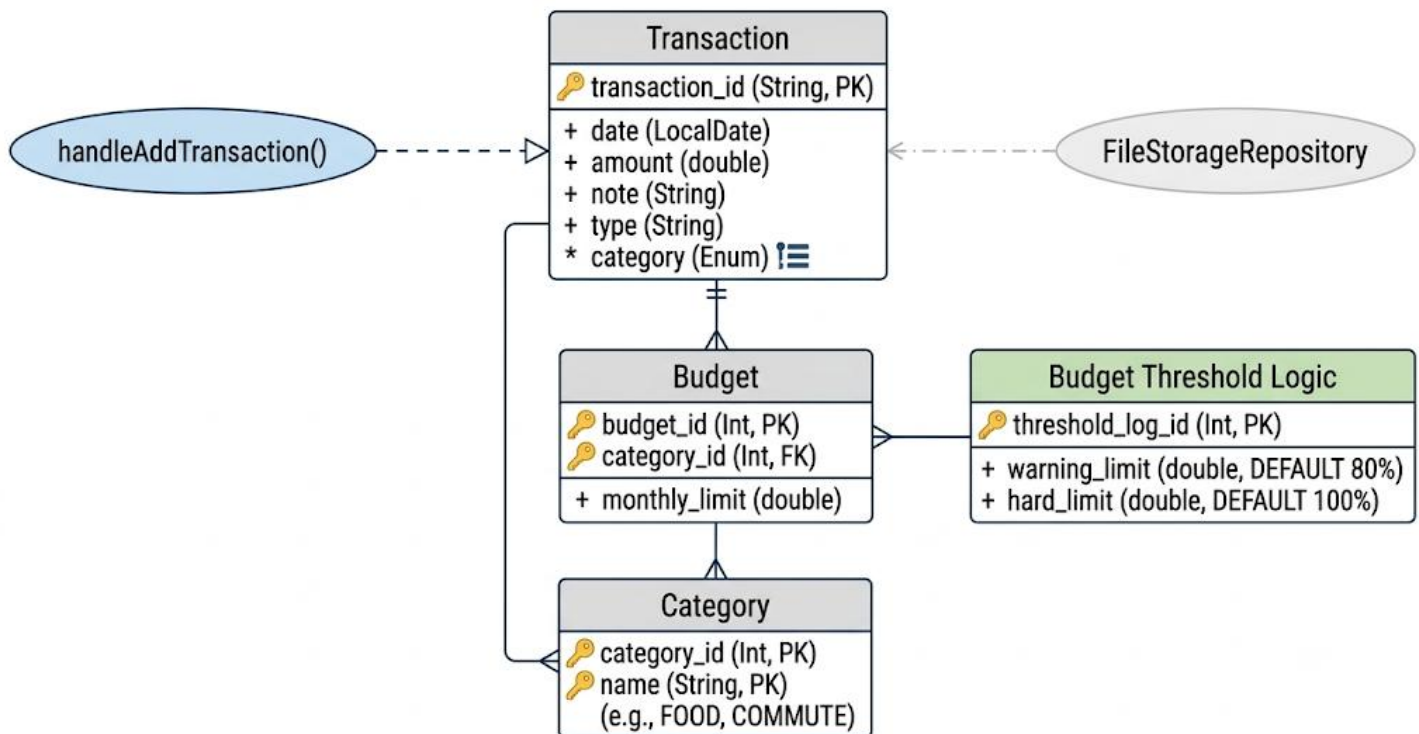
3. Sequence Diagram



4. Class Diagram



5. ER Diagram



G) Design Decisions & Rationale

- **Terminal CLI vs. Graphical UI:** A CLI architecture was chosen to eliminate heavy external dependencies (such as JavaFX or Swing libraries), ensuring platform portability, fast execution, and strict compliance with modular Java requirements.
- **Flat-File CSV Persistence over SQL Database:** Eliminates the overhead of external database server configurations (like MySQL or SQLite JDBC drivers), keeping the application lightweight, fully offline, and easily inspectable in any text editor.
- **Layered Package Structure:** Decoupling the data layer (repository) from business validation (service) and console output (ui) guarantees maintainability, testability, and adherence to clean coding principles.
- **Java Stream API for Financial Computations:** Stream filtering and mapping (mapToDouble().sum()) reduce procedural boilerplate while maximizing calculation accuracy for financial totals.
- **Two-Tier Budget Thresholding:** Implementing an 80% proactive warning alongside a 100% hard exception ensures that users receive timely notifications before funds are exhausted.

H) Implementation Details

- **Technologies Used:** Core Java (JDK 17+), Java Collections Framework (ArrayList, HashMap), Java Time API (LocalDate), and Java I/O (BufferedReader, BufferedWriter, FileWriter).
- **Defensive CSV Parsing:** The FileStorageRepository.load() method incorporates split(",", -1) to prevent dropping empty trailing tokens (such as blank transaction memos).
- **Exception Architecture:** Checked exceptions (InvalidTransactionException, BudgetExceededException) enforce strict data validation rules before state modification occurs.
- **Auto-Persistence Strategy:** Every successful write in MenuUI triggers an automated call to repository.save(), preventing data loss from unexpected terminal terminations.

I) Screenshots / Results

Figure 1: Main Menu

```
PS C:\Users\react\Personal-Expense-Tracker> java -cp bin ui.Main
[System] Initialized. Loaded 4 existing record(s) from storage.

=====
      PERSONAL EXPENSE & BUDGET TRACKER
=====
1. Log Transaction (Income/Expense)
2. View All Transactions
3. Set / View Category Budgets
4. View Financial Analytics
5. Export Statement to TXT
6. Save & Exit
Select an option (1-6): █
```

Figure 2: Transaction Logging & Threshold Alert:

```
=====
      PERSONAL EXPENSE & BUDGET TRACKER
=====
1. Log Transaction (Income/Expense)
2. View All Transactions
3. Set / View Category Budgets
4. View Financial Analytics
5. Export Statement to TXT
6. Save & Exit
Select an option (1-6): 1
Enter Type (INCOME / EXPENSE): expense
Enter Amount (?): 3500
Categories: FOOD, ACADEMICS, ENTERTAINMENT, COMMUTE, UTILITIES, SALARY, OTHER
Enter Category: food
Enter Description / Memo: party
>> Data successfully persisted to CSV.
>> Success: Transaction logged under ID [TX48641]
>> WARNING: Category FOOD is at 100.0% of budget limit! (?4000.00 / ?4000.00)
```

Figure 3: Tabular Transaction History:

```
=====
      PERSONAL EXPENSE & BUDGET TRACKER
=====

1. Log Transaction (Income/Expense)
2. View All Transactions
3. Set / View Category Budgets
4. View Financial Analytics
5. Export Statement to TXT
6. Save & Exit
Select an option (1-6): 2

----- TRANSACTION LOG -----
[TX94146] 2026-09-17 | INCOME | SALARY      | ₹2000.00 | pocket money
[TX14955] 2026-09-17 | EXPENSE | FOOD      | ₹500.00  | biriyani
[TX52649] 2026-09-17 | INCOME | SALARY      | ₹4000.00 | diwali
[TX97774] 2026-09-17 | EXPENSE | ACADEMICS  | ₹2000.00 | books
[TX48641] 2026-09-17 | EXPENSE | FOOD      | ₹3500.00 | party
```

Figure 4: Financial Analytics Overview:

```
=====
      PERSONAL EXPENSE & BUDGET TRACKER
=====

1. Log Transaction (Income/Expense)
2. View All Transactions
3. Set / View Category Budgets
4. View Financial Analytics
5. Export Statement to TXT
6. Save & Exit
Select an option (1-6): 4

===== FINANCIAL OVERVIEW =====
Total Inflow   : ₹6000.00
Total Outflow  : ₹6000.00
Net Savings    : ₹0.00
=====
```

Figure 5: Automated Test Suite Output:

```
PS C:\Users\react\Personal-Expense-Tracker> javac -d bin (Get-ChildItem -Recurse -Filter *.java src).FullName
>> java -cp bin test.TestRunner

=====
      RUNNING AUTOMATED UNIT TESTS
=====
[PASS] testAddValidTransaction
[PASS] testRejectNegativeAmount
[PASS] testNetBalanceCalculation
[PASS] testCategoryFiltering
[PASS] testBudgetExceededTrigger

-----
Tests Run: 5 | Passed: 5 | Failed: 0
=====
PS C:\Users\react\Personal-Expense-Tracker> |
```

J) Testing Approach

Test Case ID	Test Target / Module	Test Scenario & Input	Execution Method	Expected Output / Assertion	Status
TC-01	Transaction Ingestion ('ExpenseService')	Add a valid transaction ('Amount: 500.0', 'Category: FOOD', 'Type: EXPENSE', 'Note: Lunch')	Automated ('TestRunner')	Record appends successfully; 'getAllTransactions().size()' increments by 1.	 PASS
TC-02	Input Validation ('InvalidTransactionException')	Attempt to register an invalid negative monetary value ('Amount: -150.0')	Automated ('TestRunner')	Instantiation rejected; explicitly raises 'InvalidTransactionException'.	 PASS
TC-03	Financial Aggregation ('ExpenseService')	Register Inflow of '₹5000.00' and Outflow of '₹1200.00'	Automated ('TestRunner')	'getNetSavings()' returns '₹3800.00' (Inflow - Outflow).	 PASS
TC-04	Category Filtering ('ExpenseService')	Log two 'FOOD' entries ('₹300.00', '₹700.00') and one 'COMMUTE' entry ('₹250.00')	Automated ('TestRunner')	'getTotalSpentByCategory(FOOD)' accurately isolates and returns '₹1000.00'.	 PASS
TC-05	Threshold Breach Alert ('BudgetService')	Category 'FOOD' has budget ₹1000.00; cumulative spend reaches ₹1200.00	Automated ('TestRunner')	Exceeds 100% threshold; immediately triggers 'BudgetExceededException'.	 PASS

K) Challenges Faced

- **PowerShell Wildcard Compilation Failures:** Windows PowerShell does not expand path wildcards (src/*/*.java) identically to POSIX shells, causing compilation errors. This was resolved by using PowerShell's native file enumeration pipeline: `javac -d bin (Get-ChildItem -Recurse -Filter *.java src).FullName`.
- **CSV Trailing Comma Field Truncation:** Standard `String.split(",")` drops trailing empty strings if the user leaves the note/memo blank, causing `ArrayIndexOutOfBoundsException` during load cycles. This was resolved using `line.split(",", -1)`.
- **IDE Package Root Misconfigurations:** When files were initially organized inside folders with backslashes in their names or when the source root was unmapped, VS Code's Java extension flagged Incorrect Package warnings. This was resolved by organizing folders into discrete filesystem directories and properly setting the source path.

L) Learnings & Key Takeaways

- **Encapsulation & Package Modularity**: Structuring code into layered packages enforces clean separation between data storage, computation, and user interface logic.
- **Checked vs. Unchecked Exception Handling**: Designing custom checked exceptions allows domain-specific error handling to be cleanly separated from generic runtime exceptions.
- **File I/O Stream Flushing**: Gained hands-on experience with resource management, understanding how buffering mechanisms (BufferedWriter) require systematic flushing to prevent file corruption.
- **Writing Standalone Unit Test Frameworks**: Learned how to implement an automated test harness in Core Java without third-party frameworks like JUnit or Maven.

M) Future Enhancements

- **Visual Reporting**: Generating ASCII or terminal-rendered bar charts to illustrate category-wise spending proportions visually.
- **Multi-Month Historical Tracking**: Adding support for tracking historical expenses across different calendar months and generating comparative month-over-month trend analyses.
- **Encrypted Storage**: Implementing basic file encryption (e.g., AES) for transactions.csv to secure personal financial logs against unauthorized local access.
- **Search & Filter Operations**: Adding CLI search commands to query transactions by specific date ranges, keywords, or amount thresholds.

N) References

1. Oracle Java Documentation: <https://docs.oracle.com/en/java/>
2. Java SE API Specification: <https://docs.oracle.com/javase/8/docs/api/>
3. Java File I/O & BufferedReader Guide: <https://docs.oracle.com/javase/tutorial/essential/io/>
4. CSV Format Specification (RFC 4180): <https://www.ietf.org/rfc/rfc4180.txt>